



FOR SOFTWARE ENGINEERS

Bad release stopped early.

GitLab CI to AWS EKS, commit to canary: a hands-on CI/CD lab

Five repositories, one image built once, four environments, and a canary that judges itself on real orders.

Every step ran for real on GitLab and AWS while the video was recorded. Follow it in order and you end with the same running pipeline.

No AWS key is stored in GitLab. Each job trades a token GitLab signs for credentials that last minutes, and only main is allowed to.

At the time of recording, October 2026: Java 25, Node 24, Terraform 1.16. Newer versions may differ.

You build

- GitLab CI pipelines that test, scan and gate every change
- An image built once and promoted by digest through Dev, UAT, PreProd and Production on EKS
- Argo CD keeping each environment in sync with Git
- A canary that checks real orders, and alerts to Slack

Not in this lab

- Separate AWS accounts and clusters per environment
- A managed database, TLS on your own domain, real sign in
- Signed images, admission policy, SLOs from real traffic

Part 10 lists what production adds. The Kafka and GitHub versions will follow.

01 The numbers

AWS, WHILE THE CLUSTER RUNS	about 0.50 USD an hour
CAPACITY	10 vCPUs: five t3.medium nodes
THE PLATFORM APPLY	about 18 minutes, Plan: 112 to add
GITLAB CI MINUTES	about 150 to 200 of the Free plan's 400 a month
PLANTED MISTAKES	5 branches, 5 blocked merges
SAFE SCHEMA CHANGE	1,420 orders ok, 3 failed at the load balancer
AFTER THE PART 9 FIX	1,479 orders, 0 failed
CANARY STEPS	10, 25, 50, then 100 percent
THE FAULTY RELEASE	error rate 0, wrong total rate 1: aborted

From the real runs, October 2026. Read your own Terraform plan rather than chasing the count.

02 Commit to canary

One change, from a commit on main to production. The image is built once; every later environment receives the same digest by a merge request in Git, never a rebuild.

The flow, as a diagram

```
commit on main      orders-backend
|
v
verify --> sonarqube, dependency-scan, security-gate
|
v
publish      GitLab ID token --> temporary AWS credentials
              build once --> Trivy scan --> push to ECR --> digest
|
v
deploy-dev    digest into orders-gitops-dev/release.yaml
              Argo CD syncs Dev, Playwright runs against it
|
v
promote       merge request in orders-gitops, one file, digests only
              UAT --> PreProd --> Prod, the same digest each time
|
v
canary        10% --> 25% --> 50% --> 100%
              each step: enough orders, no 5xx, no wrong totals
              a failed check aborts: traffic returns to stable
```

Checkpoint 7: the same digest four times, equal to ECR.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

03 No stored AWS keys

AWS trusts tokens that gitlab.com signs, but only for one project on its protected main branch. A feature branch, a fork or another project is refused.

platform-infra: platform/gitlab-oidc.tf (excerpt, lines 1 to 35)

```
# AWS trusts tokens that gitlab.com signs. No AWS key is ever stored in GitLab:
# each CI job exchanges its own short lived ID token for temporary credentials.
resource "aws_iam_openid_connect_provider" "gitlab" {
  url           = "https://gitlab.com"
  client_id_list = ["https://gitlab.com"]
}

# One role per job that needs AWS, and each trusts exactly one project on the
# protected main branch. A feature branch, a fork or another project gets AccessDenied.
locals {
  publishers = {
    for image in local.images : image =>
    "project_path:${var.gitlab_group}/${image}:ref_type:branch:ref:main"
  }
}

data "aws_iam_policy_document" "publisher_trust" {
  for_each = local.publishers
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type       = "Federated"
      identifiers = [aws_iam_openid_connect_provider.gitlab.arn]
    }
    condition {
      test      = "StringEquals"
      variable  = "gitlab.com:aud"
      values    = ["https://gitlab.com"]
    }
    condition {
      test      = "StringEquals"
      variable  = "gitlab.com:sub"
      values    = [each.value]
    }
  }
}
```

orders-backend: .gitlab-ci.yml, the publish job (excerpt, lines 122 to 149)

```
publish:
  stage: publish
  extends: [.docker, .trivy-cache]
  image: $DOCKER_IMAGE
  needs: [verify, sonarqube, dependency-scan, security-gate]
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
  # GitLab signs a short lived token for THIS job; AWS trades it for
  # temporary credentials. No AWS key is stored anywhere in GitLab.
  id_tokens:
    AWS_ID_TOKEN:
      aud: https://gitlab.com
  variables:
    IMAGE: $ECR_REPOSITORY:$CI_COMMIT_SHORT_SHA
  script:
    # On a Kubernetes runner the docker service starts alongside the job, not
    # before it, so wait until it answers instead of failing the first command.
    - for i in $(seq 1 30); do docker info >/dev/null 2>&1 && break; sleep 2; done
    - apk add --no-cache aws-cli >/dev/null
    - >
      export $(aws sts assume-role-with-web-identity
        --role-arn "$AWS_ROLE_ARN"
        --role-session-name "gitlab-$CI_PROJECT_NAME-$CI_PIPELINE_ID"
        --web-identity-token "$AWS_ID_TOKEN"
        --duration-seconds 900
        --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' --output text
        | awk '{print "AWS_ACCESS_KEY_ID=\"$1\" AWS_SECRET_ACCESS_KEY=\"$2\" AWS_SESSION_TOKEN=\"$3\"}')
    - aws sts get-caller-identity --query Arn --output text
```

AWS_ROLE_ARN and ECR_REPOSITORY are protected project variables, filled from terraform output. Nothing in this file names an account.

Main gets temporary credentials. The demo/oidc-denied branch fails with AccessDenied ... Not authorized to perform sts:AssumeRoleWithWebIdentity.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

04 Gates that block, including when a scanner is silent

GitLab's scanners report. This job is what makes a high or critical finding block, and it fails when a report is missing: a scanner that did not run must never look like a scanner that found nothing.

Two settings decide whether a gate exists at all: `AST_ENABLE_MR_PIPELINES`: "true", without which the SAST and secret detection templates skip merge request pipelines, and `Pipelines must succeed` under Settings > Merge requests, without which a failed gate still leaves the merge button working.

```
orders-backend: .gitlab-ci.yml, the security-gate job (excerpt, lines 101 to 120)

security-gate:
  stage: quality
  image: $TOOLS_IMAGE
  needs:
    - { job: semgrep-sast, artifacts: true, optional: true }
    - { job: secret_detection, artifacts: true, optional: true }
  script:
    - apk add --no-cache jq >/dev/null
    - |
      fail=0
      for report in gl-sast-report.json gl-secret-detection-report.json; do
        # A missing report means the scanner did not run. That is a failure,
        # not a pass: silence must never look like safety.
        [ -f "$report" ] || { echo "$report is missing: the scanner did not run"; fail=1;
continue; }
        count=$(jq '[.vulnerabilities[] | select(.severity == "Critical" or .severity ==
"High")] | length' "$report")
        echo "$report: $count high or critical"
        jq -r '.vulnerabilities[] | select(.severity == "Critical" or .severity == "High") | "
\(.severity): \(.name) at \(.location.file):\(.location.start_line)'" "$report"
        [ "$count" -eq 0 ] || fail=1
      done
      exit $fail
```

demo/gate-unit-test, one cent per line: verify, expected 12498, was 12500.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

demo/gate-sast, SQL built by concatenation: security-gate, High, SQL Injection.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

demo/gate-secret, a token in a properties file: security-gate, Critical, GitLab personal access token.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

demo/gate-dependency, commons-text 1.9: dependency-scan, CVE-2022-42889, critical.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

platform-infra demo/gate-iac, port 22 open to the internet: misconfiguration-scan, AWS-0107.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

05 One artifact, four environments

Each key is allowed exactly one thing. The Dev key writes only orders-gitops-dev; the promotion key can push branches to orders-gitops but never main; Argo CD reads only. Prove it with ssh told to offer only that key.

Promotion is a merge request that copies one file. Afterwards, ask the cluster which image each environment runs.

● ● ● The Dev key, refused by the production repository (guide, step 601)

```
GIT_SSH_COMMAND="ssh -F /dev/null -o IdentitiesOnly=yes -i gitops-dev-ci" git ls-remote
git@gitlab.com:<your-group>/orders-gitops.git
# ERROR: The project you were looking for could not be found
```

● ● ● Prove nothing was rebuilt (guide, step 703)

```
for e in dev uat preprod prod; do
  kubectl -n orders-$e get pods -l app=backend -o
  jsonpath='{.items[0].status.containerStatuses[0].imageID}'; echo
done
```

promotion-check says PROMOTION, identical to what dev runs. Review the diff, digests only, and merge.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

06 A canary that judges itself

Production runs the backend as an Argo Rollouts canary. The load balancer decides the split, and at each step an analysis checks the canary's own version, never the whole service, or the healthy majority would hide it.

```
orders-gitops: charts/orders/values.yaml (excerpt, lines 34 to 36)
```

```
canary:
  steps: [10, 25, 50]  # percent of traffic, each held while analysis runs
  pause: 2m
```

```
orders-gitops: charts/orders/templates/backend.yaml, the canary strategy (excerpt, lines 46 to 58)
```

```
analysis:
  templates: [{ templateName: canary-health }]
  startingStep: 1
  args:
    - name: canary-version
      value: {{ .Values.backend.version | quote }}
    - name: namespace
      value: {{ .Release.Namespace | quote }}
  steps:
    {{- range .Values.canary.steps }}
    - setWeight: {{ . }}
    - pause: { duration: {{ $.Values.canary.pause }} }
    {{- end }}
```

orders-gitops: charts/orders/templates/monitoring.yaml, the analysis template (excerpt, lines 16 to 70)

```
# The canary passes only while its own error rate stays low. Measured on the
# canary's version label, never on the whole service, or the healthy majority
# would hide it.
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: canary-health
  labels: {{- include "orders.labels" . | nindent 4 }}
spec:
  args:
    - name: canary-version
    - name: namespace
  # "No failures yet" must read as ZERO, not as no data: a series that has
  # never had a 5xx or a wrong total does not exist, and dividing nothing by
  # something returns nothing, which Rollouts treats as an error and aborts.
  # (It aborted a healthy release exactly that way in the rehearsal.)
  metrics:
    # Enough evidence first: the canary must have served real orders.
    - name: enough-samples
      initialDelay: 1m
      interval: 1m
      count: 4
      failureLimit: 3
      successCondition: "result[0] >= 10"
      provider:
        prometheus:
          address: http://kube-prometheus-stack-prometheus.monitoring:9090
          query: |
            sum(increase(k6_checkout_checks_total{version="{{ "{{" }}args.canary-
version{{ "{{" }} }}"}}[2m])) or vector(0)
    - name: error-rate
      interval: 1m
      count: 4
      failureLimit: 1
      successCondition: "isNaN(result[0]) || result[0] < 0.02"
      provider:
        prometheus:
          address: http://kube-prometheus-stack-prometheus.monitoring:9090
          query: |
            (sum(rate(http_server_requests_seconds_count{namespace="{{ "{{" }}
args.namespace{{ "{{" }} }}"}, version="{{ "{{" }}args.canary-
version{{ "{{" }} }}"}, status=~"5..", uri=~"/api.*"[1m])) or vector(0))
            /
            (sum(rate(http_server_requests_seconds_count{namespace="{{ "{{" }}
args.namespace{{ "{{" }} }}"}, version="{{ "{{" }}args.canary-version{{ "{{" }} }}"}, uri=~"/api.*"[1m]))
or vector(0))
    # The one HTTP cannot see: of the synthetic shoppers' orders the canary
    # answered, how many had the WRONG total?
    - name: wrong-total-rate
      interval: 1m
      count: 4
```

```
failureLimit: 1
successCondition: "isNaN(result[0]) || result[0] < 0.01"
provider:
  prometheus:
    address: http://kube-prometheus-stack-prometheus.monitoring:9090
    query: |
      (sum(rate(k6_checkout_checks_total{version="{{args.canary-version{{"}}"}}",result="wrong"}[1m])) or vector(0))
      /
      (sum(rate(k6_checkout_checks_total{version="{{args.canary-version{{"}}"}}"}[1m])) or vector(0))
```

This is a Helm template. `{{"{{"}}` and `{{"}}"` print literal braces, so Argo Rollouts receives `{{args.canary-version}}` and fills in the version.

demo/faulty-total passed every gate and every environment's Playwright. In production every order it answered had the wrong total while every response was HTTP 201. The analysis failed with error rate 0 and wrong total rate 1, the rollout aborted, and traffic returned to the stable version.

Measured on real GitLab and AWS accounts while the video was recorded, October 2026. The step-by-step guide.

07 The synthetic shopper: right or wrong, per version

A k6 script orders a fixed basket whose correct total is known, and records right or wrong against the version that answered. It proves one business rule held, not that a release is healthy in every way.

orders-gitops: charts/orders/templates/synthetic.yaml, the k6 script (excerpt, lines 4 to 40)

```
import http from "k6/http";
import { Counter } from "k6/metrics";

const checks = new Counter("checkout_checks");
// Written by the init container: this environment's public address.
const BASE = "http://" + open("/shared/hostname").trim();
// A basket like real customers buy: 2 x 49.99 + 1 x 25.00 + 5 x 9.00 (people
// buy cables in fives), at the catalogue's own prices.
const ORDER = { customer: "synthetic shopper",
  lines: [{ sku: "KB-01", quantity: 2 }, { sku: "MS-02", quantity: 1 }, { sku:
"CB-04", quantity: 5 }] };
const EXPECTED_CENTS = 16998;

export const options = {
  scenarios: { shoppers: { executor: "constant-arrival-rate", rate: {{
.Values.synthetic.perSecond }}, timeUnit: "1s",
    duration: "24h", preAllocatedVUs: 10 } },
};

// Sign in the way a visitor does, then collect the CSRF token every write
// has to echo back.
export function setup() {
  const login = http.post(BASE + "/auth/demo", null, { redirects: 0 });
  const version = http.get(BASE + "/api/version");
  return { session: login.cookies.orders_session[0].value, xsrf: version.cookies["XSRF-TOKEN"]
[0].value };
}

export default function (data) {
  const res = http.post(BASE + "/api/orders", JSON.stringify(ORDER), {
    headers: {
      "Content-Type": "application/json",
      "X-XSRF-TOKEN": data.xsrf,
      Cookie: "orders_session=" + data.session + "; XSRF-TOKEN=" + data.xsrf,
    },
  });
  const version = res.headers["X-App-Version"] || "unknown";
  const right = res.status === 201 && res.json("totalCents") === EXPECTED_CENTS;
  checks.add(1, { version: version, result: right ? "right" : "wrong" });
}
```



The line that matters: a 201 is not enough. The order counts as right only if the total is also the one the basket must cost.

The repository file named above.

08 Stop the meter, and prove it

Kubernetes created some AWS resources itself: the load balancers from Ingresses and the EBS disks from volume claims. Terraform never knew about them, so the script removes them first, waits until AWS confirms they are gone, and only then runs terraform destroy. Then ask AWS directly.

From step 203 onwards, never walk away from a running cluster. Finish the checkpoint you are on, or tear down.

```
platform-infra (guide, step 1002)

cd platform-infra
export KUBECONFIG=~/.kube/cws-cicd # the script refuses any other cluster
./tools/teardown.sh                # removes what Kubernetes created, then terraform destroy
./tools/verify-teardown.sh         # asks AWS directly; every line must be 0
```

Checkpoint 10: nothing billable is running. Kept on purpose: the Terraform state bucket and the Slack webhook parameter, a free Standard SecureString.

The guide, steps 908 and 1002, and tools/verify-teardown.sh in platform-infra.

Keep beside you

1. A budget alert before anything costs money; budgets alert, they do not stop spending
2. No stored cloud keys: one role per job, trusted for one project on main
3. A missing scan report fails the gate; Pipelines must succeed is switched on
4. Build once, promote the digest; prove it with the cluster, not the pipeline
5. Judge the canary on its own version, and on the business result, not only HTTP
6. Tear down, then ask AWS directly that every line is 0

Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  GitHub
https://github.com/code-with-sam-dev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  x
https://x.com/CodeWithSamDev | orders-backend
https://gitlab.com/cws-cicd-eks/orders-backend |
|  TikTok
https://www.tiktok.com/@codewithsamdev | orders-frontend
https://gitlab.com/cws-cicd-eks/orders-frontend |
|  Instagram
https://www.instagram.com/codewithsamdev | platform-infra
https://gitlab.com/cws-cicd-eks/platform-infra |
|  Facebook
https://www.facebook.com/codewithsamdev | orders-gitops
https://gitlab.com/cws-cicd-eks/orders-gitops |
| | orders-gitops-dev
https://gitlab.com/cws-cicd-eks/orders-gitops-dev |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

GitLab is a trademark of GitLab Inc. AWS, Amazon EKS and Amazon ECR are trademarks of Amazon.com, Inc. or its affiliates. Kubernetes and Argo are trademarks of The Linux Foundation. Terraform is a trademark of HashiCorp. Grafana, Loki and k6 are trademarks of Grafana Labs. Slack is a trademark of Slack Technologies. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.